Execution-noise robustness and action-selection mismatch in a small afterstate reinforcement-learning agent for Tetris

Yerkenzhe
ISSAI Frontier Research Program 2026

29 October 2026

Abstract

We study a small afterstate reinforcement-learning (RL) agent in Tetris, asking whether it can beat a hand-written heuristic, whether training with execution noise makes it more robust to shifted placements, and whether a combo reward changes that robustness. Four agents were trained for the same 23-minute wall-clock budget: lines reward with 0% or 10% training noise, and a combo reward with 0% or 10% training noise. In a first held-out study on piece seeds 6000–6019 (1600 games), clean RL reached the 2000-piece cap (797.8 and 796.6 mean lines) and beat the rules agent (740.4). At 5% test noise the clean lines agent was best (165.4 lines), followed by clean combo (140.4), the two noise-trained agents (103.9 and 105.1), and rules (90.1). At 10% test noise the clean agents still led (46.0 and 47.3), rules reached 39.8, and both noise-trained agents reached 38.9. At 20% all five agents collapsed to 12.7–14.3 lines with overlapping intervals. The central hypothesis was not supported: under this budget, 10% training noise did not improve robustness.

After that benchmark we found an action-selection mismatch. The noise-trained agents learned an expected value over the left/intended/right execution outcomes, and acted on it during training, but the original evaluation selected placements using only the intended afterstate. A post-hoc diagnostic on the used seeds suggested that expected-noise selection could recover part of the 10% gap. We therefore ran a fresh confirmatory study on unused piece seeds 7100–7119 (2560 games), evaluating both selection modes on the same games. At 0% the modes are identical by construction (maximum absolute difference 0.0). At 5% expected selection improved clean combo by 60.2 lines (95% CI 29.0–91.2) and combo with 10% training noise by 25.4 lines (10.5–39.3); at 10% it improved lines with 10% training noise by 11.9 lines (5.4–18.6); at 20% it reduced clean lines by 3.7 lines (-5.7 to -1.9). Two diagnostic positives did not replicate on the fresh seeds. The mismatch is real and can be large, so training and evaluation backups must match. It does not, however, overturn the main negative result: at 5% noise, noise-trained agents remain far behind clean agents even under expected selection. A combo reward changes the style of play but is not a robustness intervention.

1 Introduction

Tetris is a useful test bed for sequential decision making because the action space is combinatorial but finite: for each piece the agent can enumerate rotations and columns, and the board after a placement (the *afterstate*) is easy to score. A hand-written heuristic over lines, holes, height and bumpiness is a strong baseline. The questions in this project were:

1. Can a small learned afterstate value function beat the hand-written teacher on held-out games?

2. Does training with execution noise improve performance when the same kind of noise appears at test time?

3. Does a combo reward, which values multiple simultaneous clears more than single clears, change robustness?

4. Where do the agents fail under noise, and how quickly?

The training design is a 2×2 matrix: reward (lines versus combo) crossed with training noise (0% versus 10%). Evaluation uses four test-noise rates. The final piece seeds were reserved before the final runs, and the same piece and noise streams were used for every agent.

The project produced two studies. **Study 1** is the original benchmark on piece seeds 6000–6019: five agents, 1600 games, intended-place selection for all agents. It supports a clean-play win over the rules baseline and a negative result for noise-aware training. After Study 1 we discovered that the noise-trained agents were trained to act on expected noisy outcomes but were evaluated by intended-place selection. **Study 2** is a fresh confirmatory study on unused seeds 7100–7119: four RL checkpoints, both intended and expected-noise selection, 2560 games, paired within the same seeds and noise streams. Study 2 measures the selection effect directly and tests whether the original conclusions survive a matched evaluation. We report the two studies separately and do not pool them.

Scope. This is a single-player study. All comparisons are against fixed agents on identical piece sequences; no competitive two-player duel was implemented. The agents are deliberately small: a four-feature afterstate value function, not a deep convolutional Tetris controller. The aim is a controlled robustness and evaluation study, not a state-of-the-art score.

Contributions. (i) A 1600-game held-out benchmark of reward shaping and noise-aware training, with cluster-bootstrap intervals and paired seed-level contrasts. (ii) Evidence that a train/evaluation action-selection mismatch can materially change measured robustness. (iii) A fresh 2560-game paired experiment showing where expected selection helps, where it does not, and which post-hoc diagnostic effects replicate. (iv) A complete data and code path, including per-game rows, bootstrap settings, and figure sources.

2 Related work

Value-based reinforcement learning learned strong play in backgammon [1] and, with deep networks, in Atari [2]. Afterstate values, which evaluate the board after a deterministic placement rather than the state–action pair, are a classical design for games with enumerable actions [3]. Tetris has also been attacked with cross-entropy policy search, which reached high scores with careful feature design [4]. Our agent is much smaller and our question is different: we vary reward and execution noise while holding the architecture and budget fixed.

Training under randomized dynamics is the domain-randomization idea [5]: expose the learner to perturbations during training so that it transfers to a perturbed test distribution. We apply a narrow version of this idea, a one-column execution shift at a fixed nominal rate. Our null result is specific to this budget, reward, and noise model; it is not evidence against domain randomization in general.

Because the five noise streams share a piece seed, games within a seed are not independent. We therefore resample piece seeds rather than games. Cluster bootstrap inference with few clusters is discussed by Cameron et al. [6]; we use a percentile cluster bootstrap and report intervals descriptively.

3 Experimental setup

3.1 Environment and agents

The game is a C++ Tetris engine with a 10×20 board and the seven standard tetrominoes, called from Python through a shared library. Every game has a 2000-piece cap, which corresponds to a theoretical maximum of 800 lines. The *rules* agent scores every legal placement with a fixed linear combination of four board features: cleared lines, holes, aggregate height and bumpiness.

The RL agents learn an afterstate value function. For every legal placement they compute the resulting board, extract the same four features (each scaled to roughly $[0, 1]$), and evaluate the board with a small network. The network is $4 \to 64 \to 64 \to 1$ with ReLU activations. The agent plays the highest-value place with probability $1 - \epsilon$ and a random legal place with probability ϵ; ϵ falls linearly from 1.0 to 0.01 over 30,000 pieces. Learning is afterstate temporal-difference learning with a target network, $\gamma = 0.99$, a replay memory of 50,000 transitions, mini-batches of 256, Adam with learning rate 10^{-3}, one learning step every two pieces, and a target-network copy every 500 learning steps.

The default reward is the number of cleared lines. The *combo* reward maps 0, 1, 2, 3 and 4 cleared lines to 0, 1, 3, 5 and 7. Both rewards subtract 10 when the placement ends the game. A game-over placement is assigned the value reward $- 10$ directly rather than being passed through the network.

3.2 Training runs and checkpoints

All four RL runs use piece seeds 1–990 for training, seeds 991–1000 for validation, and the same network, optimizer, exploration schedule and 23-minute wall-clock budget. The two noise-trained runs sample a placement error with probability 10% during training and use expected values over the left/intended/right outcomes in both action selection and the learning target. The noise generator is independent of the piece generator, and training game i uses stream $12000 + i$; validation is offset by 1,000,000 and uses stream $1{,}012{,}000 + i$. Both noise-trained models use the same base seed, so they meet the same execution errors.

Table 1 records the measured run sizes and the selected checkpoint. `best.pt` was selected only on the 500-piece validation games. It is not necessarily the final-game checkpoint. All four jobs exited with status 0 after about 23.0–23.1 minutes. The noisy runs saw substantially fewer pieces because noisy planning previews up to three outcomes per intended placement. The GPU model used by the queue was not recorded; evaluation ran on a 96-logical-CPU Xeon Platinum 8168 host with 1.5 TiB RAM.

3.3 Execution-noise model

With nominal probability p, the intended placement is shifted one column left or right with equal probability; rotation is unchanged. A shift into a wall or an illegal gap is blocked and the intended placement is played. Thus p is the probability that a shift is *attempted*, not the probability that the

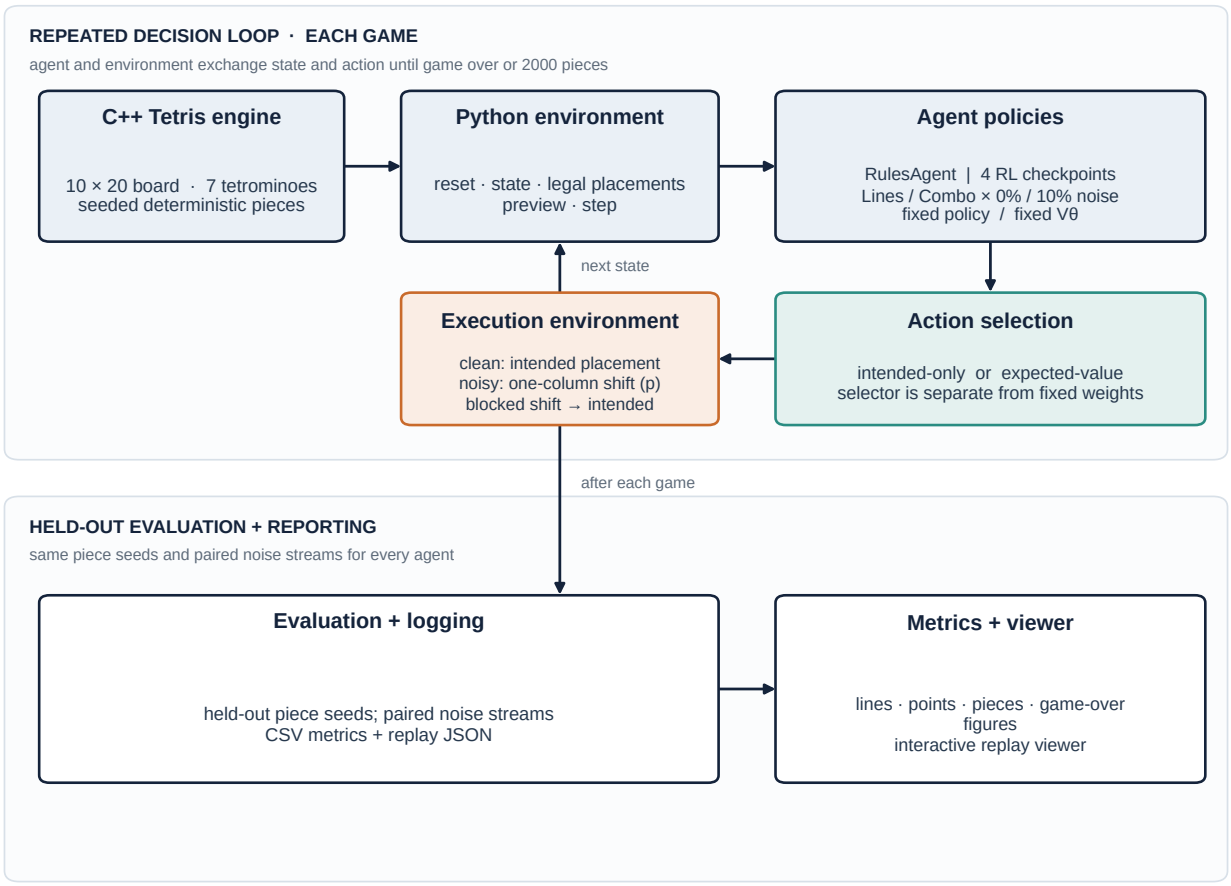


Figure 1: System pipeline. The C++ engine (10×20 board, seven tetrominoes and a seeded piece sequence) is exposed through a Python interface (`reset`, `state`, `legal_placements`, `preview`, `step`). A RulesAgent or one of four afterstate-RL checkpoints chooses an intended placement. A separate selection rule ranks placements by the intended afterstate or by the expected value over execution outcomes. The execution layer applies the placement exactly or shifts it one column with probability p, and the resulting state returns to the agent. Held-out seeds and paired noise streams drive the evaluator, which writes CSV metrics and replay JSON; analysis produces the reported metrics, figures and interactive replay viewer.

final placement differs. In Study 1, 18–21% of attempts were blocked in the noisy test conditions, so the realised shift rate was about $0.8p$. The noise generator is separate from the piece generator, so piece sequences are identical across agents and noise rates.

3.4 Intended versus expected-noise action selection

Let V be the learned afterstate value function, i an intended legal placement, and $\mathcal{O}(i, p)$ the distribution over executed outcomes: the intended placement, its legal left neighbour, and its legal right neighbour, with blocked shifts folded back into the intended outcome. The two selection rules

Table 1: Training runs and selected checkpoints. Games and pieces are those at the selected checkpoint; "Check lines" is its 500-piece validation score. The two 10%-noise runs validate under 10% noise, so their check scores are not comparable with the clean runs or with the 2000-piece final test.

Model	Reward	Train noise	Games	Pieces	Check lines
`rl1`	lines	0%	1039	121,705	198.3
`rl_combo`	combo	0%	1177	127,565	196.6
`rl_lines_noise10`	lines	10%	1562	81,103	36.9
`rl_combo_noise10`	combo	10%	1388	79,126	32.3

are

$$Q_{\text{int}}(s,i) = r(i) + \big(1 - \mathbf{1}_{\text{over}}(i)\big)\,\gamma V(s'_i) - 10\,\mathbf{1}_{\text{over}}(i), \tag{1}$$

$$Q_{\text{exp}}(s,i;p) = \sum_{o \in \mathcal{O}(i,p)} P(o \mid i,p)\Big[r(o) + \big(1 - \mathbf{1}_{\text{over}}(o)\big)\,\gamma V(s'_o) - 10\,\mathbf{1}_{\text{over}}(o)\Big]. \tag{2}$$

The intended rule ignores execution noise when ranking placements. The expected rule averages the value of the outcomes that the noise can produce. At $p = 0$ the two rules are identical by construction.

The noise-trained agents used Q_{exp} with the *training* noise rate both to act during training and to form targets. Study 1, however, evaluated every agent with Q_{int} only. That is a policy mismatch: the noise-trained checkpoints were measured with a selection rule they were not trained to use. Expected selection at evaluation time uses the evaluation rate p as a known input; it does not estimate p from observations, and it is not test-time adaptation in the sense of changing the value function.

3.5 Study 1: original held-out benchmark

Study 1 uses piece seeds 6000–6019, which were not used for training or model selection. At 0% noise each agent plays the 20 seeds once. At 5%, 10% and 20% noise each agent plays every seed with five independent noise streams, with bases 20000, 20001, 20002, 20003 and 20004; stream j for seed s uses base $20000 + j$ plus the game index. This gives 20 games per agent at 0% and 100 games per agent at each positive rate: 320 games per agent and 1600 games in total. The 0% run is stored once and used for all five agents. All five agents use intended selection.

Uncertainty is estimated with a cluster bootstrap over the 20 piece seeds (5000 repetitions, seed 20261028). Games that share a piece seed are not independent, because the five noise streams reuse the same piece sequence. Paired contrasts use the same seeds (and, at positive rates, the same five noise streams) for both agents; the bootstrap resamples seeds. The summary intervals use seed 20261028; the paired contrasts use the script's fixed offset (20261029), and the 2×2 table uses 20261030. Retention is the ratio of an agent's mean lines at a positive rate to its own mean at 0%. Intervals are percentile 95% intervals. We do not report p-values or significance stars; a contrast is described as clear only when its interval excludes zero. No multiplicity adjustment is applied, so the intervals are descriptive rather than family-wise error guarantees.

3.6 Study 2: fresh confirmatory study

Study 1 left two questions open: how much of the measured noise-training penalty was due to selection, and whether the diagnostic pattern would hold on unused seeds. Study 2 therefore uses

Table 2: The two held-out studies. Study 1 is the original benchmark, run before the action-selection mismatch was found. Study 2 is the fresh confirmatory study, run on new piece seeds with both selection modes. Both studies use the same four checkpoints, the same 2000-piece cap, and noise rates 5%, 10% and 20%.

Study	Piece seeds	Models	Games	Noisy games	Selection
1, original benchmark	6000–6019	5	100	1500	intended
2, fresh confirmatory	7100–7119	4	160	2400	intended vs expected

Table 3: Study 1 (original benchmark): mean lines cleared per game on the untouched final seeds 6000–6019 (2000-piece cap). The 10% column also gives the 95% cluster-bootstrap interval over the 20 piece seeds. At 0% one stream is used; at 5–20% five noise streams are averaged inside each seed, so games are not independent.

Model	Train noise	0%	5%	10% (95% CI)	20%
RulesAgent	–	740.4	90.1	39.8 [36.8, 43.1]	14.3
Lines RL, 0% train noise	0%	797.8	165.4	46.0 [42.5, 49.7]	13.4
Combo RL, 0% train noise	0%	796.6	140.4	47.3 [42.6, 53.0]	13.1
Lines RL, 10% train noise	10%	797.6	103.9	38.9 [35.0, 43.0]	13.4
Combo RL, 10% train noise	10%	765.9	105.1	38.9 [35.0, 42.7]	12.7

piece seeds 7100–7119, which were never used in training, validation, checkpoint selection or Study 1. It evaluates the four RL checkpoints under both selection modes on the same games. At 0% each model and mode plays 20 seeds once (160 games). At 5%, 10% and 20% each model and mode plays every seed with five streams, with bases 21000, 21001, 21002, 21003 and 21004; stream j for seed s uses base $21000 + j$ plus the game index. This gives 100 games per model, mode and positive rate, and 2560 games in total. The 0% rows for the two modes are duplicated runs and are identical by construction; the analysis verifies exact parity, and the maximum absolute difference is 0.0 for lines, points and pieces.

Study 2 uses the same four checkpoints as Study 1, with no retraining. It has no rules baseline: rules comparisons remain Study 1 results. The fresh design is paired for selection (intended versus expected on the same seed and noise stream) but was not powered for a new clean-versus-noise-trained comparison. Bootstrap settings are 5000 repetitions with seed 20261029 for summary intervals and the script's fixed offset (20261036) for paired intervals, again resampling the 20 piece seeds. Table 2 summarises the two studies.

4 Results

4.1 Study 1: clean play

At 0% test noise three RL agents reached the cap in all 20 games: 797.8 (lines, 0% train), 796.6 (combo, 0% train) and 797.6 (lines, 10% train) lines, out of a theoretical 800. The rules agent averaged 740.4 lines and ended 4 of 20 games early. The combo, 10%-noise agent reached the cap in 19 of 20 games and averaged 765.9 lines; this single early game explains its wide interval [702.2, 798.1]. Table 3 gives the means.

The paired rules–RL contrast at 0% is clear: rules minus lines-trained RL is −57.35 lines (95% CI −114.25 to −11.74). The clean lines and combo agents differ by only 1.1 lines at 0% (CI 0.25–1.9), a small gap relative to the cap; both are at the cap. Clean play therefore does not meaningfully

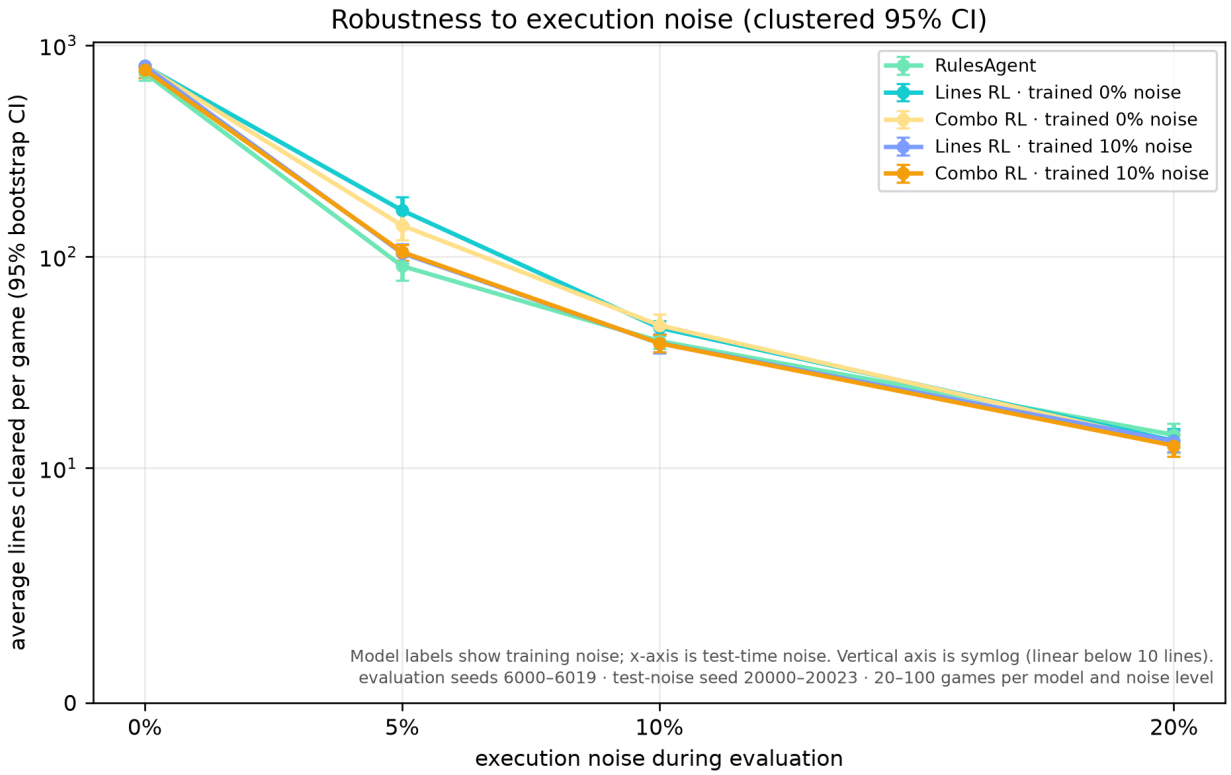


Figure 2: Study 1: mean lines per game against test-time execution noise. Vertical bars are 95% cluster-bootstrap intervals over the 20 piece seeds; the vertical axis is symmetric-log (linear below 10 lines). Labels state the training noise; the x-axis states the test noise.

separate the two reward functions; it mainly shows that all RL agents beat the teacher. The combo agent nevertheless accumulates more points (110,145 versus 93,980 per game) because the combo reward values multi-line clears, even though the line total is capped.

4.2 Study 1: robustness to execution noise

Performance collapses quickly. At 5% nominal noise the best agent, clean lines RL, clears 165.4 lines (about 20.7% of its clean score). The clean combo agent clears 140.4, the noise-trained agents 103.9 and 105.1, and the rules agent 90.1. At 10% noise the clean agents lead with 46.0 and 47.3 lines, rules reaches 39.8, and both noise-trained agents reach 38.9. At 20% noise every agent is between 12.7 and 14.3 lines, and all intervals overlap. Figure 2 shows the full curves on a symmetric-log axis so that both the near-cap clean scores and the small noisy scores are visible. Table 4 gives retention relative to each agent's own clean score.

The ranking at 5% is clear and consistent with the paired intervals in Table 5: clean lines RL is 25.0 lines above clean combo RL (CI 6.8–42.9), 61.5 above lines RL trained with noise (39.6–84.2), and 60.3 above combo RL trained with noise (38.5–84.2). Clean combo RL is 36.5 lines above lines RL trained with noise (18.2–58.3). The two noise-trained agents are indistinguishable (1.2 lines, CI −8.9 to 12.4). At 10%, clean lines RL is 7.1 lines above its noise-trained counterpart (2.7–11.2), and clean combo RL is 8.4 lines above its noise-trained counterpart (3.1–15.0). The two noise-trained agents are again indistinguishable (0.0 lines, CI −5.1 to 4.5). At 20% every paired interval contains

Table 4: Study 1 (original benchmark): retention of lines cleared relative to the same model at 0% test noise (mean ratio; 95% cluster-bootstrap interval). A value of 1.0 means no loss.

Model	5% noise	10% noise	20% noise
RulesAgent	0.122 [0.104, 0.140]	0.054 [0.049, 0.060]	0.019 [0.017, 0.022]
Lines RL, 0% train noise	0.207 [0.177, 0.239]	0.058 [0.053, 0.062]	0.017 [0.015, 0.019]
Combo RL, 0% train noise	0.176 [0.152, 0.205]	0.059 [0.053, 0.067]	0.016 [0.015, 0.018]
Lines RL, 10% train noise	0.130 [0.117, 0.144]	0.049 [0.044, 0.054]	0.017 [0.015, 0.019]
Combo RL, 10% train noise	0.137 [0.124, 0.153]	0.051 [0.044, 0.059]	0.017 [0.014, 0.019]

Table 5: Study 1 (original benchmark): paired differences in lines per game. Each cell is Δ lines [95% cluster-bootstrap interval]; positive means the first named model is better. Seeds are shared, and positive-rate rows also share the five noise streams.

Comparison	5% test noise	10% test noise	20% test noise
Combo − Lines, both 0% train	-25.0 [-42.9, -6.8]	1.3 [-4.0, 6.6]	-0.4 [-2.1, 1.4]
0% − 10% train, Lines reward	61.5 [39.6, 84.2]	7.1 [2.7, 11.2]	0.1 [-1.9, 2.1]
0% − 10% train, Combo reward	35.3 [17.7, 58.7]	8.4 [3.1, 15.0]	0.3 [-1.2, 1.8]
Combo − Lines, both 10% train	1.2 [-8.9, 12.4]	0.0 [-5.1, 4.5]	-0.6 [-2.5, 1.3]

zero. The rules agent matches the noise-trained RL agents at 10% (+0.9 and +1.0 lines, intervals including zero), while both clean agents remain above rules.

4.3 Study 1: reward shaping

The combo reward changes the *style* of clean play without changing the line total. Figure 3 shows that the clean combo agent clears far fewer singles (238.9 versus 540.8 per game) and more doubles, triples and tetrises (211.2 versus 92.9 doubles; 31.6 versus 16.9 triples; 10.2 versus 5.2 tetrises). Because the 2000-piece cap truncates clean play, this does not translate into more lines. At 5% noise the clean lines agent is better on lines, while the combo agent still produces more multi-line clears; at 10% the two clean agents are statistically indistinguishable (1.3 lines, CI −4.0 to 6.6). The combo reward is therefore not a robustness intervention in this setup.

4.4 Study 1: the 2×2 design and training confound

Table 6 and Figure 4 show the four trained cells at 0% and 10% test noise. At 0% the cells are all near the cap; the combo, 10%-noise cell is lower because one of its 20 games ended early. At 10% test noise the two clean-trained cells retain 5.8–5.9% of their clean score, while the two noise-trained cells retain 4.9–5.1%. Training with 10% noise therefore did not improve retention; if anything, it reduced it in this budget. The clean-minus-noisy differences are 7.1 lines for the lines reward (CI 2.7–11.2) and 8.4 lines for the combo reward (CI 3.1–15.0).

Figure 5 shows the training curves and the 500-piece validation checks. The clean models approach the cap on the validation games. The noise-trained models look much lower on validation because those checks are played under 10% noise; their validation score is not a clean-play score and must not be compared with the 2000-piece final test. The training-game totals also differ because the budget is wall-clock time: the noisy runs saw 81,103 and 79,126 pieces, versus 121,705 and 127,565 for the clean runs, about 33–38% fewer. This is a confound for any claim that noise-aware training is harmful in general. What the experiment does show is that, under this budget, the noise-trained checkpoints were not more robust than the clean-trained ones.

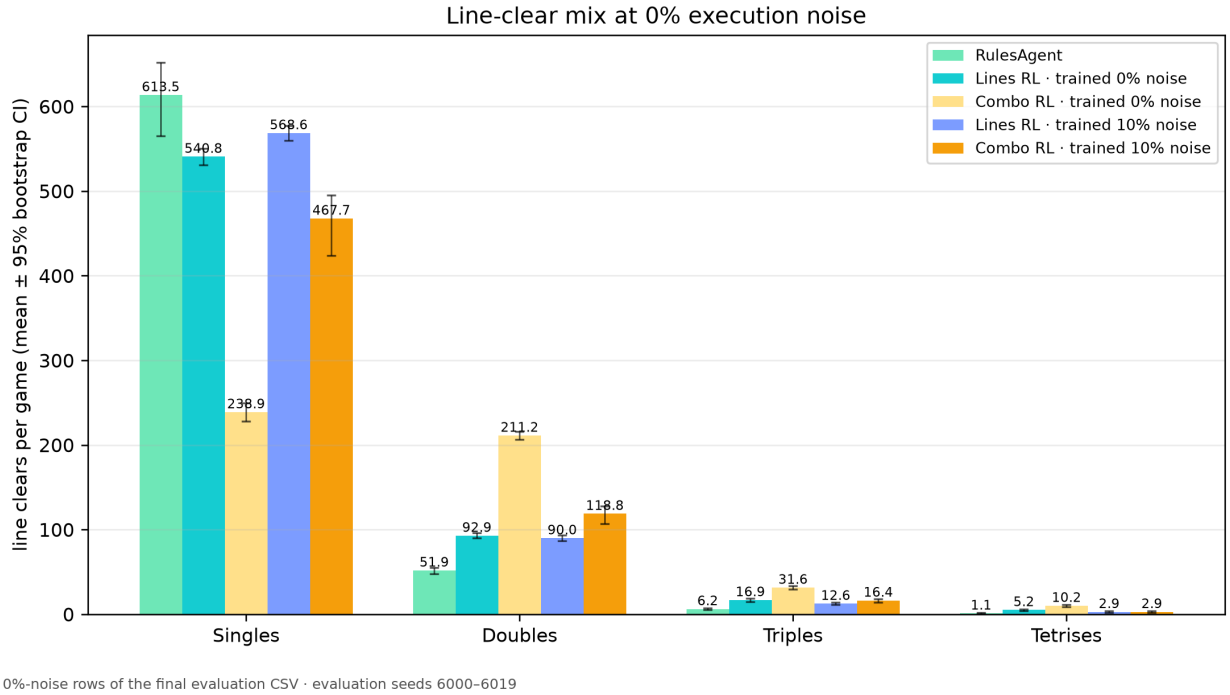


Figure 3: Study 1: line-clear mix at 0% test noise, in clears per game (mean and 95% cluster-bootstrap interval). The combo reward substitutes doubles, triples and tetrises for singles but is capped by the same 2000-piece limit.

Figure 6 shows retention for the two rewards and the two training-noise conditions on one axis. It makes the main negative result visible directly: at 5% noise both clean models retain more than either noisy model, and at 10% the same ordering holds, though the gap is small.

4.5 Study 1: survival under 10% noise

Figure 7 gives Kaplan–Meier survival curves at 10% noise. Every one of the 500 games ended below the 2000-piece cap; the last death was at 530 pieces. The clean lines agent stays slightly above the others between about 150 and 300 pieces, but the curves are close and the experiment was not powered to resolve small survival differences. One game stopped at 253 pieces because no legal placement remained while the engine's game-over flag stayed false; we count any stop below the cap as a death, not as censoring. Across Study 1, three additional 5% games stopped below the cap with the flag false. Treating all four as deaths changes no mean-line table and affects one game in the 10% survival panel. No game was censored at the cap.

4.6 Study 1: post-hoc selection diagnostic

The selection mismatch was found only after Study 1. To estimate its size, we re-evaluated the two noise-trained checkpoints on the *used* Study 1 seeds 6000–6019 and noise base 20000, once with intended selection and once with expected selection at the same test rate. This diagnostic is post hoc and cannot be confirmatory. It gave two positive cells: lines with 10% training noise at 5% test noise improved by +22.5 lines (95% CI 4.1–43.4), and combo with 10% training noise at 10% test noise improved by +17.8 lines (1.9–38.1). The other two cells were positive but uncertain: lines

Table 6: Study 1 (original benchmark): 2×2 design; reward (lines vs combo) by training noise (0% vs 10%), measured at 0% and 10% test noise. Cells give mean lines [95% cluster-bootstrap interval] and retention relative to the same model at 0% test noise.

Test noise	Reward	Train noise	Mean lines	Retention
0%	Combo	0%	796.6 [796.1, 797.1]	–
0%	Combo	10%	765.9 [702.2, 798.1]	–
0%	Lines	0%	797.8 [797.2, 798.3]	–
0%	Lines	10%	797.6 [796.9, 798.1]	–
10%	Combo	0%	47.3 [42.6, 53.0]	0.059
10%	Combo	10%	38.9 [35.0, 42.7]	0.051
10%	Lines	0%	46.0 [42.5, 49.7]	0.058
10%	Lines	10%	38.9 [35.0, 43.0]	0.049

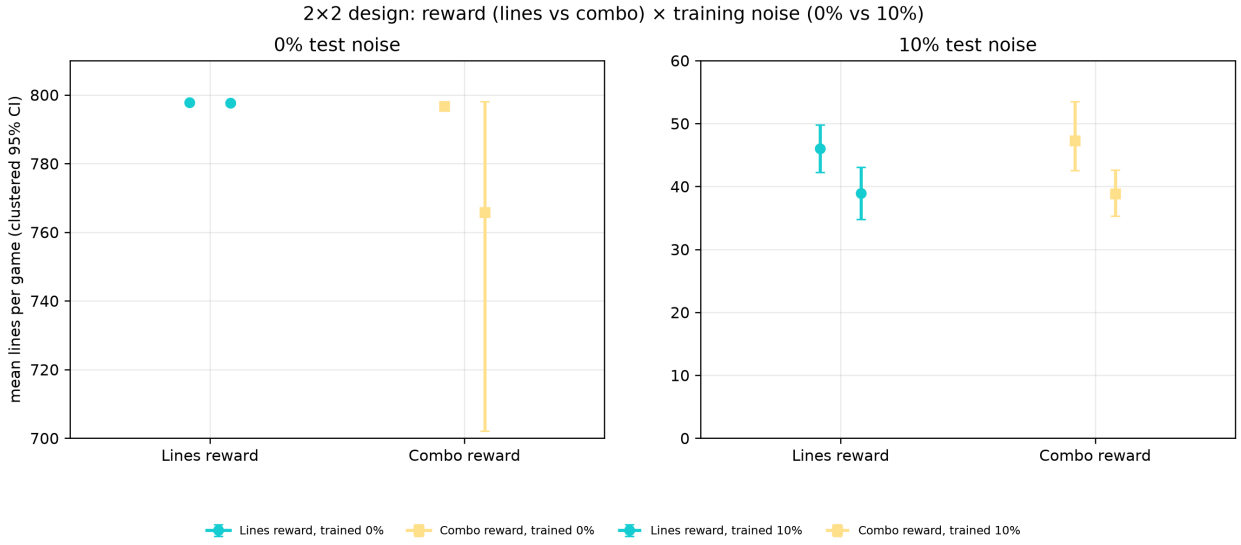


Figure 4: Study 1: 2 × 2 design at 0% (left, 700–810 lines) and 10% (right, 0–60 lines) test noise. Points are means; bars are clustered 95% bootstrap intervals. The same 20 piece seeds are used in every cell; the 10% cells use five shared noise streams.

with 10% training noise at 10% was +3.4 (−11.6 to 18.6), and combo with 10% training noise at 5% was +15.1 (−33.0 to 67.5). These values motivated Study 2 but are not independent evidence.

4.7 Study 2: fresh intended-versus-expected experiment

Table 7 and Figure 8 give the fresh results. At 0% the two modes are identical, as expected: all four models are at the cap (796.8–797.8 lines), and the paired difference is exactly 0.0. At 5% expected selection is clearly better for clean combo (+60.2 lines, CI 29.0–91.2) and for combo with 10% training noise (+25.4, 10.5–39.3). The effects for clean lines (+25.2, CI −21.0 to 67.7) and lines with 10% training noise (+15.4, CI −3.5 to 35.4) are positive but not resolved. At 10% expected selection is clearly better for lines with 10% training noise (+11.9, CI 5.4–18.6); the other three cells are small and uncertain (combo clean +1.3, combo noise +0.7, lines clean −0.9). At 20% expected selection is worse for clean lines (−3.7, CI −5.7 to −1.9); the other three cells are near zero with intervals including zero.

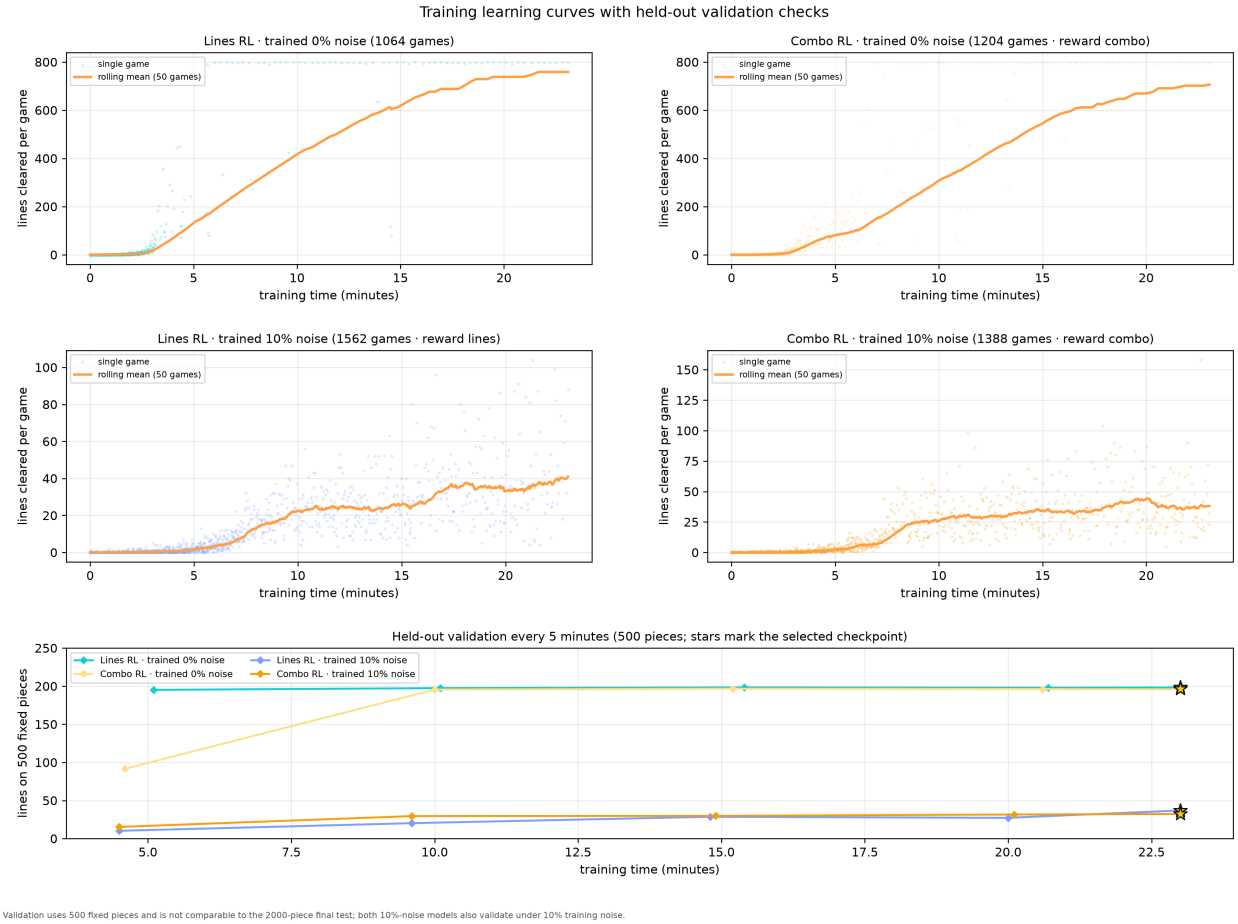


Figure 5: Training of the four checkpoints used in both studies. Top: lines per training game (dots) and a 50-game rolling mean. Bottom: 500-piece validation checks every five minutes; stars mark the selected checkpoints. Both 10%-noise models validate under 10% noise, so their validation panel is not comparable with clean play or with the final 2000-piece test.

The two diagnostic positives did not replicate. Lines with 10% training noise at 5% moved from +22.5 (4.1–43.4) in the diagnostic to +15.4 (−3.5 to 35.4) on fresh seeds; combo with 10% training noise at 10% moved from +17.8 (1.9–38.1) to +0.7 (−6.6 to 8.1). Conversely, the fresh study found two positives that the diagnostic did not: clean combo at 5% and combo with 10% training noise at 5%. The general lesson is therefore not that a particular cell is robust, but that the selection rule can change the measured ranking in both directions.

The fresh study also bounds the practical importance of the mismatch. At 5%, even with expected selection, the noise-trained agents clear 118.9 and 116.4 lines, far below clean lines (203.4) and clean combo (171.4). The negative result at 5% therefore survives a matched evaluation. At 10%, the raw means under expected selection are 47.4 (clean lines), 46.7 (clean combo), 50.9 (lines, 10% training noise) and 44.4 (combo, 10% training noise); the clean-versus-noisy contrast is no longer clear in these descriptive means, but Study 2 was not designed or powered for that contrast, so we do not claim that noise training helps at 10%. At 20%, all four expected-selection means are 11.6–14.4 lines and the differences are small.

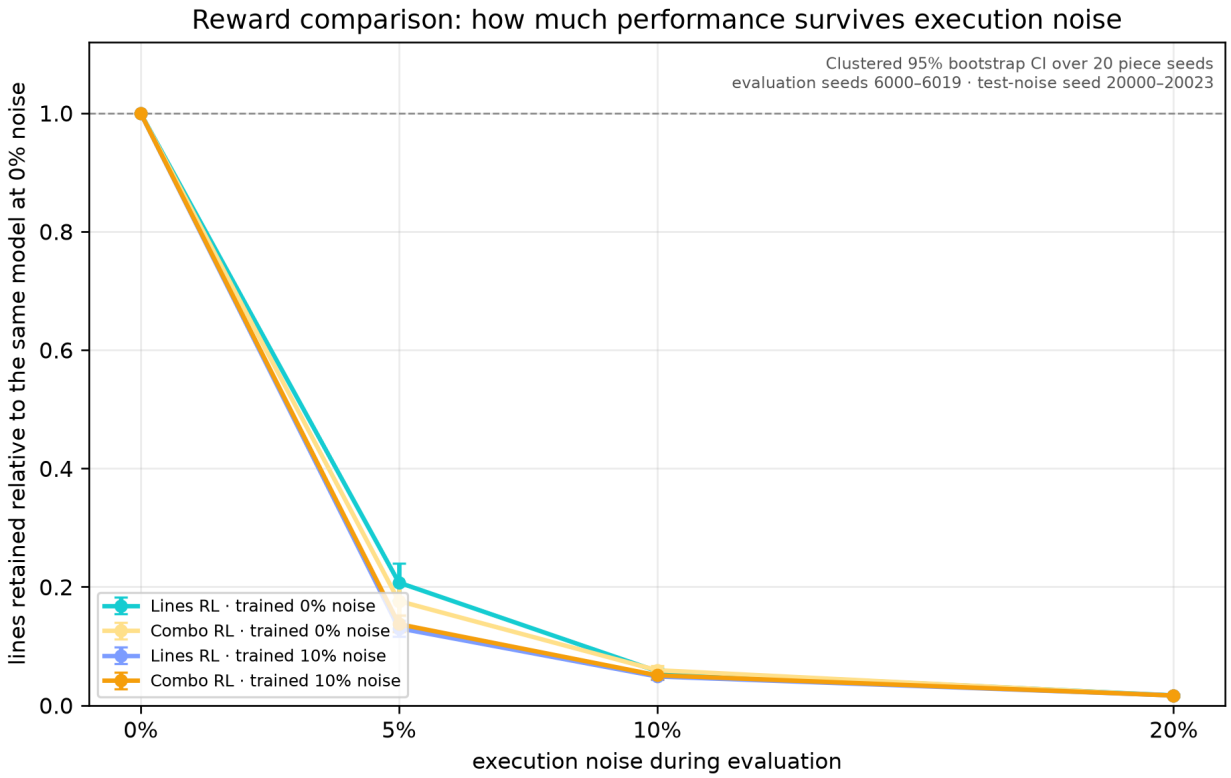


Figure 6: Study 1: retention of lines cleared relative to the same model at 0% test noise. Intervals are clustered 95% bootstrap intervals over the 20 piece seeds. A value of 1.0 is no loss; the dashed line marks 1.0.

5 Discussion

The clean result is unambiguous: a four-feature afterstate RL agent beats the four-feature hand-written teacher on the held-out seeds, and the line total is limited only by the 2000-piece cap. The noise result is the opposite of the motivating hypothesis. Under a fixed 23-minute budget, a 10% training-noise rate did not produce a more robust policy at 5% test noise. At 20% noise all agents are equally bad, so there is no room for a robustness advantage to appear. At 10%, the original intended-selection comparison showed a 7–8-line clean advantage; the fresh expected-selection means are close, but that contrast is not the one Study 2 was designed to test.

There are three plausible explanations for the negative training result, and this experiment does not separate them cleanly. First, the expected-value target may smooth the value function in a way that slows learning of the precise placements that matter. Second, the noisy runs had about 35% fewer pieces of experience under the fixed wall-clock budget, because each planning step previewed up to three outcomes; less experience alone could explain the gap. Third, 10% training noise may be too high for a four-feature value function: the agent learns to expect errors, but the test distribution at 5% is different and the policy is not adaptive at test time.

The action-selection mismatch is a separate finding. The original benchmark used a legitimate policy—intended-place selection—but it was not the policy that the noise-trained agents optimized. Expected selection can change the measured mean by tens of lines at 5% noise, and it can also hurt: clean lines at 20% lost 3.7 lines. A likely interpretation is that expected selection hedges

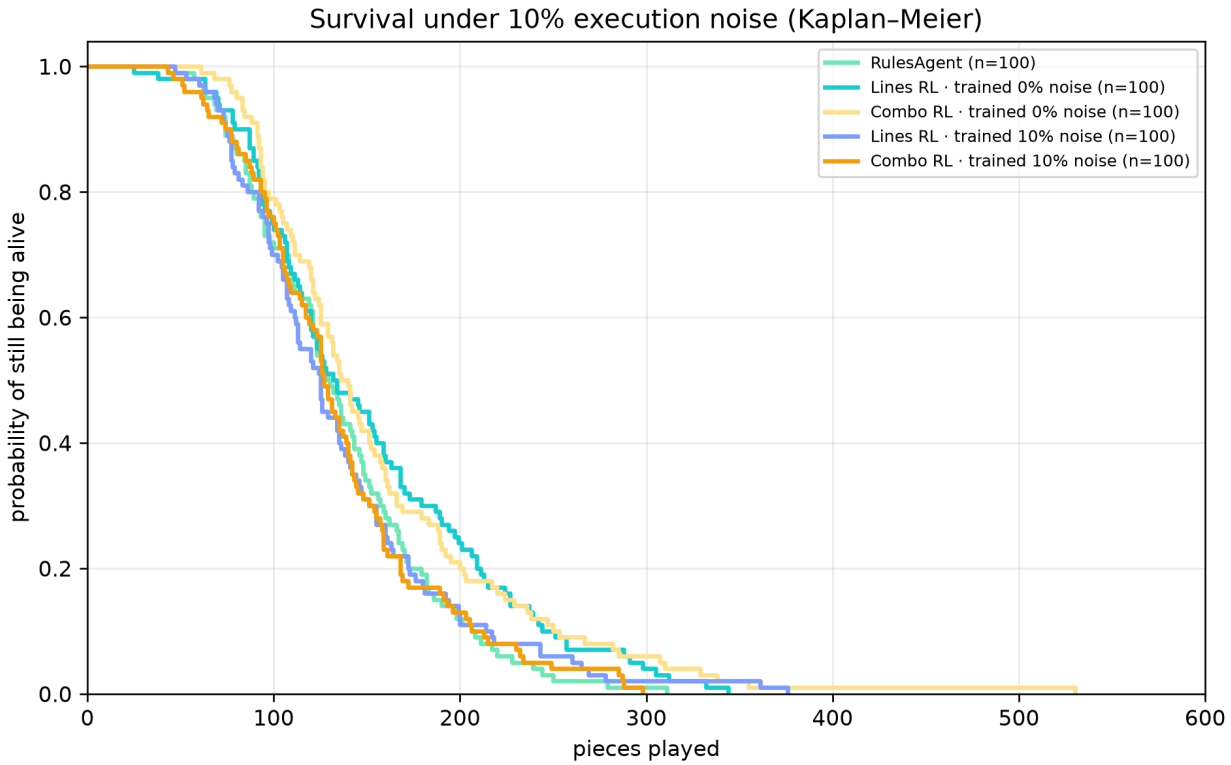


Figure 7: Study 1: Kaplan–Meier survival at 10% test noise, 100 games per agent (20 seeds × 5 streams). A stop below the 2000-piece cap is treated as a death, including one game that ended with no legal placement.

toward outcomes with lower variance, which helps when noise is moderate and the value function is uncertain, but can sacrifice line-clearing at very high noise. We did not test that interpretation directly.

The combo reward is a style intervention, not a robustness intervention. It reduces singles and increases multi-line clears at 0% noise, and it produces more points, but it does not improve line retention under noise. At 10% test noise the two clean rewards are indistinguishable. The large fresh gain for clean combo at 5% shows that the combo policy's ranking is especially sensitive to the selection rule, not that combo training is more robust.

For practice, the main lesson is that an evaluation policy must be stated and matched to training. If the learner acts on expected noisy outcomes, evaluating it on intended outcomes measures a different policy. Reporting both modes, as in Study 2, is cheap and informative. The second lesson is that robustness claims need a matched budget: with a wall-clock budget, noise-aware planning buys fewer pieces of experience, and that confound can dominate the effect of noise training. A cleaner test would give the noisy runs a piece budget rather than a time budget, match training and test noise rates, train at several noise levels, and estimate the test noise rate rather than supplying it as an oracle.

6 Limitations and threats to validity

- **Twenty piece seeds.** The cluster bootstrap has 20 clusters. Intervals for small differences are wide, and the 20% comparisons are underpowered. The 5% and 10% clean-versus-noisy

Table 7: Fresh confirmatory study on piece seeds 7100–7119: mean lines cleared per game under intended and expected-noise action selection, and the paired difference Δ (expected minus intended) with its 95% cluster-bootstrap interval. At 0% the two modes are identical by construction and empirically (maximum absolute difference 0.0); positive rates use 20 seeds and five shared noise streams. Intervals resample the 20 piece seeds (5000 repetitions).

Model	Test noise	Intended	Expected	Δ [95% CI]
Lines RL, clean	0%	797.8	797.8	0.0 [0.0, 0.0]
	5%	178.2	203.4	+25.2 [-21.0, 67.7]
	10%	48.3	47.4	-0.9 [-7.4, 6.0]
	20%	15.3	11.6	-3.7 [-5.7, -1.9]
Combo RL, clean	0%	796.8	796.8	0.0 [0.0, 0.0]
	5%	111.1	171.4	+60.2 [29.0, 91.2]
	10%	45.4	46.7	+1.3 [-4.2, 7.0]
	20%	13.2	13.1	-0.1 [-1.2, 1.1]
Lines RL, 10% training noise	0%	796.9	796.9	0.0 [0.0, 0.0]
	5%	103.6	118.9	+15.4 [-3.5, 35.4]
	10%	38.9	50.9	+11.9 [5.4, 18.6]
	20%	14.5	14.2	-0.4 [-2.4, 1.7]
Combo RL, 10% training noise	0%	797.6	797.6	0.0 [0.0, 0.0]
	5%	91.0	116.4	+25.4 [10.5, 39.3]
	10%	43.7	44.4	+0.7 [-6.6, 8.1]
	20%	14.4	14.4	0.0 [-1.8, 1.8]

intervals exclude zero in Study 1, but the exact effect size is uncertain.

- **One training run per cell.** Each of the four checkpoints is a single run. Training-seed variance is not estimated, so the four checkpoints are not four independent estimates of a training method.

- **Wall-clock training budget.** The noisy runs saw fewer pieces. The comparison therefore estimates the performance of checkpoints produced by this budget, not the effect of noise-aware training with equal experience.

- **One noise model.** Noise is an independent one-column shift with a wall/gap block. Rotation errors, multi-column shifts, and correlated errors are not tested.

- **Validation saturation.** The 500-piece validation cap is close to the maximum possible score (about 200 lines), so it cannot rank clean models finely. The noisy models validate under 10% noise, which makes their check scores incomparable with clean validation.

- **Engine flag.** Four Study 1 games stopped below the cap without the engine setting its game-over flag (three at 5%, one at 10%). We treat any stop below the cap as a death. This changes one game in the 10% survival panel and no mean-line table.

- **No test-time adaptation.** The agents do not estimate the noise rate or adapt their value function at test time. Expected selection is given the evaluation p as an oracle.

- **Study 2 is not a full replication.** It has no rules baseline and was powered for paired selection contrasts, not for a new clean-versus-noise-trained test. The fresh clean means differ slightly from Study 1 because the piece seeds differ.

- **Descriptive intervals.** No multiplicity adjustment is applied. We use intervals to describe uncertainty, not to make family-wise significance claims.

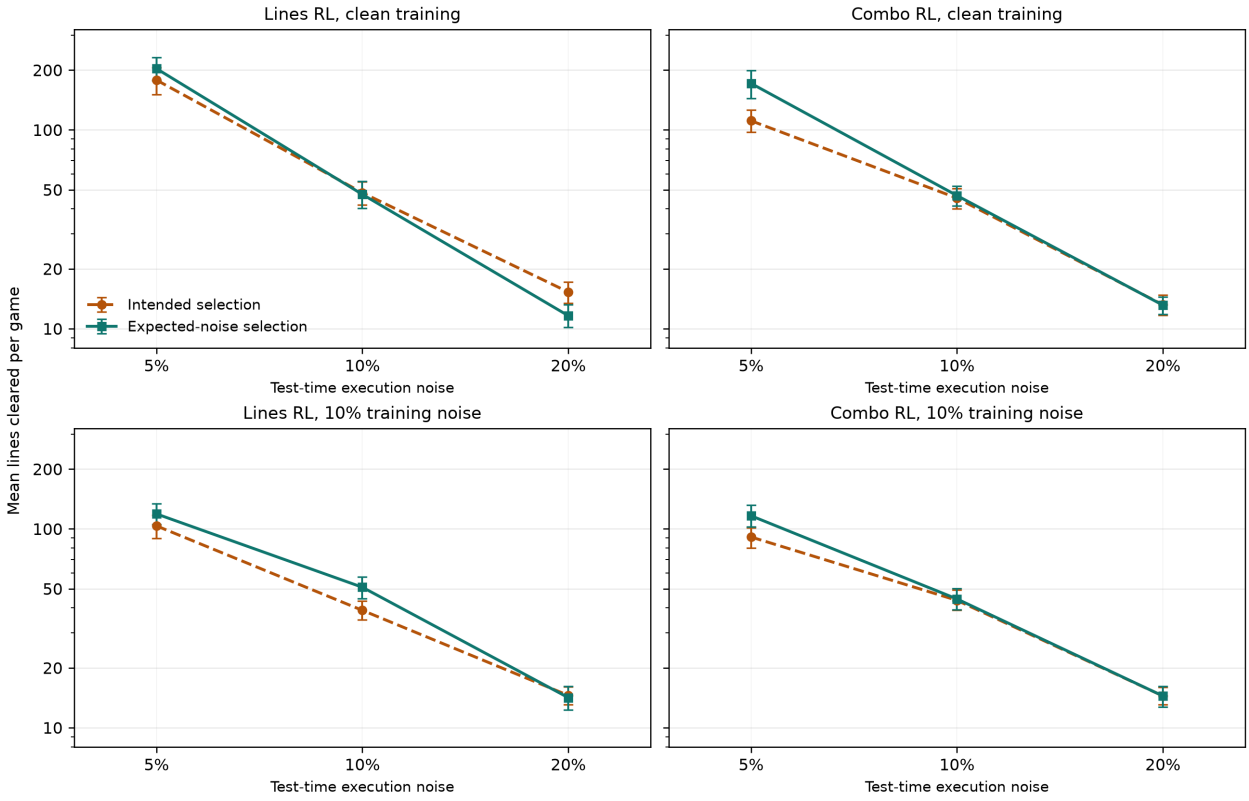


Figure 8: Study 2: fresh piece seeds 7100–7119. Intended selection (dashed) and expected-noise selection (solid) at 5%, 10% and 20% test noise; 20 seeds × 5 shared streams in every cell. Vertical bars are 95% clustered bootstrap intervals over the 20 piece seeds. The vertical axis is logarithmic. At $p = 0$ the two modes are identical (0.0 difference) and are omitted.

7 Reproducibility

All evaluation rows, summaries, paired contrasts and the 2×2 table for Study 1 are generated from the six batch CSVs in `final_batches/` by `finalize_results.py`. The protected exploratory files are not overwritten. The six Study 1 figures are generated by `plot_research.py` from the four training logs, the four validation CSVs, the final per-game CSV and the 0%-noise detailed CSV; `figures/manifest.json` records the inputs, bootstrap settings, smoothing window and output list. The Study 1 LaTeX tables are generated by `make_tables.py` from the final CSVs.

Study 2 is in `policy_mismatch/`. The raw games are `full/` and `p0/`; `analyze.py` produces `results_policy_mismatch_summary.csv`, `results_policy_mismatch_paired.csv` and `results_policy_mismatc` The manifest records the seed range, noise bases, five streams, 5000 bootstrap repetitions, seed 20261029 and the cluster unit (piece seed). The fresh table is a direct transcription of the summary and paired CSVs; Figure 8 plots the mean and interval columns of the summary CSV. The regression tests in `tests/test_policy_mismatch_analysis.py` cover checkpoint-path normalisation and p=0 parity; the full suite has 66 passing tests.

Six full-game replays for seed 6000 are in `replays/`: five at 0% noise (rules, both clean RL agents, and both noise-trained agents) and one combo game at 10% noise with the intended-placement ghost and execution-error markers. The repository is `https://github.com/Yerkenje/tetris-rl-arena`; the results in this manuscript correspond to commit `bf75e7c`. The manuscript source is `main.tex`

with `references.bib`, `tables/` and `figures/`; it is compiled with pdfLaTeX and BibTeX.

8 AI-writing disclosure

The author used the `opencode` command-line assistant with the model DeepSeek-V4.1-Flash to help develop code, run experiments, perform the statistical analysis, generate tables and figures, and draft and edit this manuscript. The assistant is not an author. Every number in the tables and text comes from the committed scripts and CSV files listed above; the author checked the aggregates, the paired contrasts and the replay exports, and takes responsibility for the content.

References

[1] Gerald Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995. doi: 10.1145/203330.203343.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018. ISBN 9780262039246.

[4] Christophe Thiéry and Bruno Scherrer. Improvements on learning Tetris with cross entropy. *ICGA Journal*, 32(1):23–33, 2009. doi: 10.3233/ICG-2009-32104.

[5] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017. doi: 10.1109/IROS.2017.8202133.

[6] A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3):414–427, 2008. doi: 10.1162/rest.90.3.414.